

Introduction to SQL

Q-1 What is SQL, and why is it essential in database management?

Ans : SQL (Structured Query Language) is a standard language used to interact with relational databases. It allows users to create, read, update, and delete data stored in tables, as well as manage the structure and security of the database.

SQL is used with databases like:

MySQL

Oracle

PostgreSQL

SQL Server

SQLite

Like we are using many type of language we are using

1– DDL (create , alter , drop , rename)

2– DML (insert , delete , update)

3 – DCL (grant , revoke)

4– TCL (rollback , save point , commit)

5 – DQL (select)

Q-2 Explain the difference between DBMS and RDBMS.

Ans :

DBMS (Database Management System)

A DBMS is software that allows users to store, retrieve, and manage data in a database. Data is generally stored as files or in a non-relational structure.

Key Features of DBMS

- Data stored in file format
- No relation between tables/files
- Supports single user mostly
- Less security
- Redundancy (duplicate data) possible
- No foreign key / primary key concept

Examples

- File System

- XML-based DBMS
- FoxPro
- MS Access (basic usage)

RDBMS (Relational Database Management System)

An **RDBMS** is an advanced form of DBMS where data is stored in **tables (relations)**, and tables are **connected using keys**.

Key Features of RDBMS

- Data stored in **table (row & column) format**
- Supports **relationships** between tables
- **Multi-user support**
- High **data security**
- **No redundancy** due to normalization
- Supports **Primary Key & Foreign Key**
- Supports **ACID properties**

Examples

- MySQL
- Oracle
- SQL Server
- PostgreSQL

Q-3. Describe the role of SQL in managing relational databases.

Ans : SQL manages relational databases using different categories of commands. Each category has a specific role in database management.

1. DDL (Data Definition Language)

Definition:

DDL is used to define, create, and modify the structure of database objects in a relational database.

Includes:

- CREATE – Used to create database objects such as tables and databases.
- ALTER – Used to modify the structure of an existing table.
- DROP – Used to delete database objects permanently.
- TRUNCATE – Used to remove all records from a table while keeping its structure.

Role: Manages the database structure.

2. DML (Data Manipulation Language)

Definition:

DML is used to insert, update, and delete data in relational tables.

Includes:

- INSERT – Adds new records into a table.
- UPDATE – Modifies existing records in a table.
- DELETE – Removes records from a table.

Role: Manages the data stored in tables.

3. DQL (Data Query Language)

Definition:

DQL is used to retrieve data from one or more relational tables.

Includes:

- SELECT – Fetches required data using conditions and joins.

Role: Retrieves meaningful information from the database.

4. DCL (Data Control Language)

Definition:

DCL is used to control access and permissions in the database.

Includes:

- GRANT – Provides access rights to users.
- REVOKE – Removes access rights from users.

Role: Ensures database security.

5. TCL (Transaction Control Language)

Definition:

TCL is used to manage transactions and maintain data consistency.

Includes:

- COMMIT – Saves all changes permanently.
- ROLLBACK – Undoes changes made during a transaction.
- SAVEPOINT – Creates a point to roll back partially.

Role: Maintains data consistency and reliability.

Q-4 What are the key features of SQL?

Ans :

1. Data Definition Feature

Definition:

SQL allows users to define and manage the structure of database **objects**.

Includes:

- CREATE – Used to create tables, databases, and other objects.
 - ALTER – Used to modify the structure of existing tables.
 - DROP – Used to delete database objects permanently.
 - TRUNCATE – Used to remove all records from a table while keeping its structure.
-

2. Data Manipulation Feature

Definition:

SQL provides commands to insert, update, and delete data in relational tables.

Includes:

- INSERT – Adds new records into a table.
 - UPDATE – Modifies existing records.
 - DELETE – Removes records from a table.
-

3. Data Retrieval Feature

Definition:

SQL enables users to retrieve required data from one or more tables.

Includes:

- SELECT – Fetches data using conditions, joins, and sorting.
-

4. Relationship Management Feature

Definition:

SQL supports relationships between tables in relational databases.

Includes:

- **PRIMARY KEY** – Uniquely identifies each record.
 - **FOREIGN KEY** – Establishes relationships between tables.
-

5. Data Integrity Feature

Definition:

SQL ensures accuracy and consistency of data using constraints.

Includes:

- **NOT NULL** – Prevents null values.
 - **UNIQUE** – Avoids duplicate values.
 - **CHECK** – Validates data conditions.
-

6. Transaction Management Feature

Definition:

SQL maintains data consistency using transaction control.

Includes:

- **COMMIT** – Saves changes permanently.
 - **ROLLBACK** – Cancels changes.
 - **SAVEPOINT** – Allows partial rollback.
-

7. Security Feature

Definition:

SQL provides security and access control to protect data.

Includes:

- **GRANT** – Gives permissions to users.
- **REVOKE** – Removes permissions from users.

SQL Syntax

1) What are the basic components of SQL syntax?

1. Clauses

Definition:

Clauses are keywords in SQL statements that define conditions or actions.

Includes:

- **SELECT** – Specifies the columns to retrieve.
- **FROM** – Specifies the table(s) from which to retrieve data.
- **WHERE** – Filters records based on conditions.
- **GROUP BY** – Groups records based on one or more columns.
- **HAVING** – Filters groups based on conditions.
- **ORDER BY** – Sorts the result in ascending or descending order.

Role: Determines what data to retrieve and how.

2. Expressions

Definition:

Expressions are combinations of operators, column names, and values that produce a result.

Includes:

- **Arithmetic operators:** +, -, *, /
- **Comparison operators:** =, <, >, <=, >=, <>
- **Logical operators:** AND, OR, NOT

Role: Performs calculations or comparisons in SQL queries.

3. Predicates

Definition:

Predicates are conditions that evaluate to TRUE, FALSE, or UNKNOWN to filter data.

Includes:

- **BETWEEN** – Checks if a value is within a range.
- **IN** – Checks if a value matches any value in a list.
- **LIKE** – Checks a pattern in text.
- **IS NULL** – Checks for null values.

Role: Filters records in queries.

4. Queries

Definition:

A query is a statement that retrieves data from a database.

Includes:

- **SELECT statements**
- Can include joins, subqueries, and aggregations

Role: Retrieves required data efficiently.

5. Statements

Definition:

Statements are complete SQL commands that perform a task.

Includes:

- **DDL statements (CREATE, ALTER, DROP)**
- **DML statements (INSERT, UPDATE, DELETE)**

- TCL statements (COMMIT, ROLLBACK)
- DCL statements (GRANT, REVOKE)

Role: Executes specific actions on the database.

6. Clauses, Keywords, and Identifiers

Definition:

SQL syntax uses keywords (reserved words), identifiers (names of tables/columns), and clauses to form valid commands.

Includes:

- **Keywords:** SELECT, FROM, WHERE
- **Identifiers:** employee, salary, dept_id
- **Clauses:** ORDER BY, GROUP BY, HAVING

Role: Ensures correct structure and execution of SQL commands.

Q-2 Write the general structure of an SQL SELECT statement.

Ans :

1. SELECT Clause

Definition:

Specifies the columns to retrieve from a table.

Includes:

- * – Selects all columns.
- Column names – Select specific columns like name, salary.

Example:

SELECT name, salary

Role: Determines which data to fetch.

2. FROM Clause

Definition:

Specifies the table(s) from which to retrieve data.

Includes:

- **Single table** – FROM employees

- **Multiple tables** – using JOIN

Example:

FROM employees

Role: Indicates the source table(s) for the data.

3. WHERE Clause (Optional)

Definition:

Filters records based on conditions.

Includes:

- Comparison operators: =, >, <
- Logical operators: AND, OR, NOT

Example:

WHERE salary > 30000

Role: Retrieves only the records that meet the condition.

4. GROUP BY Clause (Optional)

Definition:

Groups records based on one or more columns.

Example:

GROUP BY department

Role: Organizes data for aggregate functions like SUM, COUNT.

5. HAVING Clause (Optional)

Definition:

Filters groups created by GROUP BY based on a condition.

Example:

HAVING COUNT(*) > 5

Role: Filters aggregated data.

6. ORDER BY Clause (Optional)

Definition:

Sorts the result in ascending (ASC) or descending (DESC) order.

Example:

ORDER BY salary DESC

Role: Determines the order of output.

General Structure (Complete Syntax)

SELECT column1, column2, ...

FROM table_name

WHERE condition

GROUP BY column1, column2, ...

HAVING condition

ORDER BY column1 ASC|DESC;

Q-3 Explain the role of clauses in SQL statements.

Ans :

1. SELECT Clause

Definition:

Specifies the columns or expressions to retrieve from a table.

Includes:

- * – Selects all columns.
- Column names – Select specific columns like name, salary.

Role: Determines which data to fetch.

2. FROM Clause

Definition:

Specifies the table(s) from which to retrieve data.

Includes:

- Single table – FROM employees

- **Multiple tables – using JOIN operations**

Role: Indicates the source table(s) of data.

3. WHERE Clause (Optional)

Definition:

Filters records based on specified conditions.

Includes:

- **Comparison operators:** =, >, <
- **Logical operators:** AND, OR, NOT

Role: Retrieves only the records that satisfy the condition.

4. GROUP BY Clause (Optional)

Definition:

Groups records based on one or more columns for aggregation.

Includes:

- Used with aggregate functions like SUM(), COUNT(), AVG()

Role: Organizes data for summary calculations.

5. HAVING Clause (Optional)

Definition:

Filters groups of records created by GROUP BY based on a condition.

Includes:

- Used with aggregate functions like SUM() > 10000

Role: Filters aggregated data.

6. ORDER BY Clause (Optional)

Definition:

Sorts the result in ascending (ASC) or descending (DESC) order.

Includes:

- **Single or multiple columns:** ORDER BY salary DESC, name ASC

Role: Determines the order of the output.

SQL Constraints

Q-1 What are constraints in SQL? List and explain the different types of constraints ?

Ans :

Types of SQL Constraints (Sequential) and Their Definitions

1. **NOT NULL**
Definition: Ensures that a column **cannot have a NULL value**; every row must have a value.
2. **UNIQUE**
Definition: Ensures that all values in a column are **distinct**; no duplicates are allowed.
3. **PRIMARY KEY**
Definition: Combines **NOT NULL** and **UNIQUE** to **uniquely identify each record** in a table.
4. **FOREIGN KEY**
Definition: Ensures **referential integrity** by linking a column in one table to a **primary key in another table**.
5. **CHECK**
Definition: Ensures that values in a column **meet a specified condition** (rule).
6. **DEFAULT**
Definition: Assigns a **default value** to a column when no value is provided during insertion.

Q-2 How do PRIMARY KEY and FOREIGN KEY constraints differ?

Ans :

Difference Between PRIMARY KEY and FOREIGN KEY

1. **Definition:**
 - **PRIMARY KEY:** Uniquely identifies each record in a table.
 - **FOREIGN KEY:** Ensures referential integrity by linking a column to the primary key of another table.
2. **Uniqueness:**
 - **PRIMARY KEY:** Values must be **unique**.

- **FOREIGN KEY:** Values may **repeat**, but must match values in the referenced table.
3. **NULL Values:**
- **PRIMARY KEY:** Cannot have NULL values.
 - **FOREIGN KEY:** Can be NULL (unless restricted).
4. **Purpose:**
- **PRIMARY KEY:** Identifies each row **uniquely**.
 - **FOREIGN KEY:** Maintains **relationship** between two tables.
5. **Applied On:**
- **PRIMARY KEY:** Column(s) in the **same table**.
 - **FOREIGN KEY:** Column(s) in a **child table** referencing the parent table.
6. **Example:**
- **PRIMARY KEY:** emp_id INT PRIMARY KEY
 - **FOREIGN KEY:** dept_id INT FOREIGN KEY REFERENCES department(dept_id)

Q-3 What is the role of NOT NULL and UNIQUE constraints?

Ans :

1. NOT NULL Constraint

Definition: Ensures that a column cannot have a NULL value; every row must have a value.

Role: Guarantees that essential data is always entered and prevents missing or empty values in the column.

Example:

```
CREATE TABLE employee (
    emp_id INT NOT NULL,
    name VARCHAR(50) NOT NULL
);
```

2. UNIQUE Constraint

Definition: Ensures that all values in a column are distinct; no duplicates are allowed.

Role: Maintains uniqueness of data in a column, ensuring no repeated values.

Example:

```
CREATE TABLE employee (
    email VARCHAR(50) UNIQUE
```

);

NOT NULL ensures that mandatory fields always have values, while UNIQUE ensures that column values are distinct, both helping to maintain data integrity in a table.

Main SQL Commands and Sub-commands(DDL)

Q-1 Define the SQL Data Definition Language (DDL)?

Ans :

Definition:

SQL Data Definition Language (DDL) is a subset of SQL used to define, create, modify, and delete database structures such as databases, tables, indexes, and schemas. DDL commands deal with the structure of the database, not the data itself.

Main DDL Commands

1. CREATE

- Definition: Used to create database objects like tables, views, or databases.
- Example:

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary INT  
);
```

2. ALTER

- Definition: Used to modify the structure of an existing database object.
- Example:

```
ALTER TABLE employee ADD COLUMN dept_id INT;
```

3. DROP

- Definition: Used to delete database objects permanently.
- Example:

```
DROP TABLE employee;
```

4. TRUNCATE

- Definition: Removes all data from a table but keeps its structure intact.
- Example : TRUNCATE TABLE employee;

Q-2 Explain the CREATE command and its syntax.

Ans : The CREATE command is a Data Definition Language (DDL) command used to create database objects such as databases, tables, views.

It defines the structure of the object, not the data inside it.

Create the database :

Syntax : CREATE DATABASE database_name;

Create table :

Syntax : CREATE TABLE table_name (
 column1 datatype,
 column2 datatype,
 column3 datatype
);

Create view :

Syntax : CREATE VIEW view_name AS
 SELECT column1, column2
 FROM table_name;

Q-3 . What is the purpose of specifying data types and constraints during table creation?

Ans :

1. Purpose of Data Types

Data types define the kind of data that can be stored in a column.

Why data types are important:

- Ensure that only **valid data** is stored in each column
- Improve **data accuracy and consistency**
- Optimize **storage space**
- Help the database perform **faster processing and calculations**

Example:

- INT → stores numeric values
 - VARCHAR(50) → stores text up to 50 characters
 - DATE → stores date values
-

2. Purpose of Constraints

Constraints are rules applied to columns to maintain **data integrity** in a table.

Why constraints are important:

- Prevent **invalid or duplicate data**
 - Enforce **business rules**
 - Maintain **accuracy and reliability** of data
-

Common Constraints and Their Purpose

- **PRIMARY KEY:**
Ensures each record is **unique** and **not null**
- **NOT NULL:**
Prevents a column from having empty values
- **UNIQUE:**
Prevents duplicate values in a column
- **FOREIGN KEY:**
Maintains relationship between tables
- **CHECK:**
Ensures values satisfy a condition
- **DEFAULT:**
Assigns a default value if no value is provided

ALTER Command

Q-1 What is the use of the ALTER command in SQL?

Ans : The **ALTER command** is a **Data Definition Language (DDL)** command used to **modify the structure of an existing database object**, mainly a table.

It allows changes to be made **after the table has been created**, without deleting the table.

Uses of ALTER Command

The ALTER command is used to:

- Add a new column to a table
- Modify an existing column's data type or size
- Delete (drop) a column from a table
- Rename a column or table (in some databases)
- Add or remove constraints (PRIMARY KEY, FOREIGN KEY, etc.)

Q-2 How can you add, modify, and drop columns from a table using ALTER?

Ans :

1. Add a Column .

```
ALTER TABLE student  
ADD age INT;
```

2. Modify a Column .

```
ALTER TABLE student  
MODIFY name VARCHAR(100);
```

3. Drop a Column .

```
ALTER TABLE student  
DROP COLUMN age;
```

4. Add a Constraint .

```
ALTER TABLE student  
ADD PRIMARY KEY (roll_no);
```

6-DROP Command

Q-1 What is the function of the DROP command in SQL?

Ans :

Definition

The DROP command is a Data Definition Language (DDL) command used to permanently remove database objects from the database.

When an object is dropped, **both its structure and data are deleted permanently**.

What DROP Command Can Remove

The DROP command can be used to delete:

- Database
- Table
- View

Q-2 What are the implications of dropping a table from a database?

Ans :

1. Permanent Data Loss

- All **rows (records)** stored in the table are **deleted permanently**
 - The data **cannot be recovered** unless a backup exists
-

2. Table Structure is Removed

- The table definition (columns, data types, constraints) is completely removed
 - The table no longer exists in the database
-

3. Loss of Constraints and Indexes

- All **primary keys, foreign keys, indexes, and triggers** related to the table are deleted
 - Relationships with other tables are broke
-

4. Impact on Dependent Objects

- Views, stored procedures, or queries that depend on the table may **fail or become invalid**
 - Applications using the table may stop working
-

5. Cannot Be Rolled Back

- In most databases, **DROP cannot be undone**
- Transaction rollback does **not** restore a dropped table

Data Manipulation Language (DML)

Q-1 Define the INSERT, UPDATE, and DELETE commands in SQL.

Ans :

INSERT, UPDATE, and DELETE Commands in SQL

These commands are **Data Manipulation Language (DML)** commands used to **manage data** stored in database tables.

1. INSERT Command

Definition

The **INSERT command** is used to **add new records (rows)** into a table.

Syntax

```
INSERT INTO table_name (column1, column2, ...)
```

```
VALUES (value1, value2, ...);
```

Example

```
INSERT INTO student (roll_no, name, marks)
```

```
VALUES (1, 'Amit', 85);
```

Purpose

- Adds new data to a table
-

2. UPDATE Command

Definition

The **UPDATE command** is used to **modify existing records** in a table.

Syntax

```
UPDATE table_name
```

```
SET column1 = value1, column2 = value2
```

```
WHERE condition;
```

Example

```
UPDATE student
```

```
SET marks = 90
```

WHERE roll_no = 1;

Purpose

- Changes existing data
 - WHERE clause specifies which records to update
-

3. DELETE Command

Definition

The **DELETE command** is used to **remove records** from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Example

DELETE FROM student

WHERE roll_no = 1;

Purpose

- Removes selected rows from a table
 - Without WHERE, all rows are deleted
-

Key Points

- INSERT adds data
- UPDATE modifies data
- DELETE removes data
- All are **DML commands**

Q-2 What is the importance of the WHERE clause in UPDATE and DELETE operations?

Ans :

Definition

The **WHERE clause** is used in **UPDATE** and **DELETE** statements to **specify which rows** should be modified or removed.

Why the WHERE Clause is Important

1. Prevents Accidental Data Loss

- Without a WHERE clause:
 - **UPDATE** modifies **all rows**
 - **DELETE** removes **all rows**
- This can cause **serious and irreversible data loss**

2. Allows Targeted Changes

- The WHERE clause ensures that **only specific records** meeting a condition are affected
- Helps apply changes **accurately and safely**

3. Maintains Data Integrity

- Prevents unintended changes to unrelated data
- Ensures correctness and reliability of the database

Example

UPDATE with WHERE

UPDATE student

SET marks = 95

WHERE roll_no = 1;

→ Updates marks of **only one student**

DELETE with WHERE

DELETE FROM student

WHERE marks < 40;

→ Deletes **only students who failed**

Without WHERE (Dangerous)

DELETE FROM student;

→ Deletes **all records** from the table

Data Query Language (DQL)

Q-1 What is the SELECT statement, and how is it used to query data?

Ans : The SELECT statement is an SQL command used to get data from a database table. It helps us see the data stored in the table.

SELECT = show data

How SELECT is Used

Basic Syntax :

SELECT column_name

FROM table_name;

Easy Examples

1. Show All Data

SELECT * FROM student;

Shows all rows and all columns from the student table.

2. Show Selected Columns

SELECT name, marks FROM student;

Shows only name and marks.

3. Show Data with Condition

SELECT * FROM student WHERE marks > 60;

Shows only students who scored more than 60 marks.

4. Sort Data

SELECT * FROM student ORDER BY marks;

Shows data in sorted order.

Q-2. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

Ans :

The **WHERE** and **ORDER BY** clauses are used in SQL queries to **filter and arrange data** in a clear and meaningful way.

The **WHERE clause** is used to **select specific rows** from a table based on a given condition. It helps display only the required data and ignores unnecessary records. For example, it can be used to show only those students who scored more than a certain number of marks.

The **ORDER BY clause** is used to **sort the result of a query** in either ascending or descending order. By default, data is sorted in ascending order, but it can also be sorted in descending order using the DESC keyword. This makes the output easier to read and analyze.

When both clauses are used together, the **WHERE clause first filters the data**, and then the **ORDER BY clause sorts the filtered data**.

Example:

```
SELECT * FROM student
```

```
WHERE marks > 60
```

```
ORDER BY marks DESC;
```

Data Control Language (DCL)

Q-1 What is the purpose of GRANT and REVOKE in SQL?

Ans :

Purpose of GRANT and REVOKE in SQL

The GRANT and REVOKE commands are used in SQL to control user access and permissions on database objects such as tables, views, and databases.

The GRANT command is used to give permissions to users or roles. It allows users to perform specific actions like SELECT, INSERT, UPDATE, DELETE, or ALL on database objects. This helps ensure that users can access only the data they are authorized to use.

The REVOKE command is used to remove permissions that were previously given using the GRANT command. It prevents users from performing certain actions on database objects, thereby improving database security.

By using GRANT and REVOKE, a database administrator can protect data, control access, and maintain security in the database.

1] GRANT SYNTAX :

GRANT privileges_name

ON table_name

TO user_name;

2] REVOKE SYNTAX :

REVOKE privilege_name

ON table_name

FROM user_name;

Q-2 . How do you manage privileges using these commands?

Ans :

Managing Privileges Using GRANT and REVOKE Commands

Privileges in SQL are managed by **giving and removing permissions** on database objects using the **GRANT** and **REVOKE** commands.

The **GRANT command** is used to **assign specific privileges** to a user or role, such as the ability to read, insert, update, or delete data from a table. This ensures that users can perform only the operations they are allowed to do.

The **REVOKE command** is used to **withdraw previously granted privileges** when access is no longer required or needs to be restricted. This helps prevent unauthorized use of data.

By carefully granting only required permissions and revoking unnecessary ones, database administrators can **control access, maintain data security, and reduce misuse of database resources**.

Example:

GRANT SELECT, INSERT ON student TO user1;

REVOKE INSERT ON student FROM user1;

Transaction Control Language (TCL)

Q-1 What is the purpose of the COMMIT and ROLLBACK commands in SQL?

Ans :

The **COMMIT** and **ROLLBACK** commands are used in SQL to **manage transactions** and ensure **data consistency** in a database.

1. COMMIT

Purpose:

- The COMMIT command is used to **save all changes** made in the current transaction permanently to the database.
- Once committed, the changes **cannot be undone**.

Example:

```
UPDATE student
```

```
SET marks = 90
```

```
WHERE roll_no = 1;
```

COMMIT;

- The marks of the student are now **permanently saved** in the database.

2. ROLLBACK

Purpose:

- The ROLLBACK command is used to **undo all changes** made in the current transaction.
- It restores the database to its **previous state** before the transaction started.

Example:

```
UPDATE student
```

```
SET marks = 90
```

```
WHERE roll_no = 1;
```

ROLLBACK;

- The marks of the student are **not changed**; the database remains as it was before the update

Q-2 Explain how transactions are managed in SQL databases.

Ans :

Managing Transactions in SQL Databases

A **transaction** in SQL is a **sequence of one or more SQL operations** performed as a single logical unit. Transactions ensure that the database remains **consistent, accurate, and reliable**, even in case of errors, failures, or multiple users accessing the database simultaneously.

Key Properties of Transactions – ACID

Transactions are managed using the **ACID properties**:

1. **Atomicity** – All operations in a transaction must **either complete fully or not at all**.
Example: If an UPDATE fails, the entire transaction is undone.
2. **Consistency** – The database must remain in a **valid state** before and after the transaction.
3. **Isolation** – Transactions run independently without interfering with each other.
4. **Durability** – Once a transaction is **committed**, the changes are permanent, even in case of system failure.

SQL Joins :

Q-1 . Explain the concept of JOIN in SQL. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

Ans :

Concept of JOIN in SQL

A JOIN in SQL is used to combine data from two or more tables based on a common column between them, usually a primary key in one table and a foreign key in another table.

Types of JOIN in SQL

1. INNER JOIN

- Returns **only those records that have matching values in both tables**.
- Rows without a match in either table are **not included**.

Example:

If a student exists in both Student and Marks tables, only those students are shown.

2. LEFT JOIN (LEFT OUTER JOIN)

- Returns **all records from the left table**.
 - Also returns matching records from the right table.
 - If there is no match in the right table, the result shows **NULL** values.
-

3. RIGHT JOIN (RIGHT OUTER JOIN)

- Returns **all records from the right table**.
 - Also returns matching records from the left table.
 - If there is no match in the left table, the result shows **NULL** values.
-

4. FULL OUTER JOIN

- Returns **all records from both tables**.
 - Matching rows are combined.
 - Non-matching rows contain **NULL** values where data is missing.
-

Difference Between JOIN Types

JOIN Type	Description
INNER JOIN	Returns only matching records from both tables
LEFT JOIN	Returns all records from left table and matching records from right table
RIGHT JOIN	Returns all records from right table and matching records from left table
FULL OUTER JOIN	Returns all records from both tables

Q-2 How are joins used to combine data from multiple tables?

Ans :

How are joins used to combine data from multiple tables?

Joins are used in SQL to **retrieve related data from two or more tables** by linking them through a **common column**, usually a **primary key** in one table and a **foreign key** in another.

Explanation

- When data is stored in multiple tables, related information is split to avoid duplication.
- A **JOIN condition** is written in the SQL query to specify **how the tables are connected**.
- SQL then matches rows from the tables based on this condition and displays the combined result.

Key Points

- Joins combine tables **horizontally** (by columns).
- The common column is used to match related records.
- Different join types control **which rows are included** in the result.

SQL Group By

Q-1 . What is the GROUP BY clause in SQL? How is it used with aggregate functions?

Ans :

What is the GROUP BY Clause in SQL?

The **GROUP BY** clause in SQL is used to **group rows that have the same values in one or more columns** and treat each group as a single unit.

It is commonly used **with aggregate functions** to perform calculations on each group of data.

Aggregate Functions Used with GROUP BY

Aggregate functions perform calculations on a set of values and return **one single result**.

Common aggregate functions are:

- **COUNT()** – counts rows
- **SUM()** – adds values
- **AVG()** – calculates average
- **MAX()** – finds maximum value
- **MIN()** – finds minimum value

How GROUP BY Works

- The table data is divided into groups based on the column specified in GROUP BY.

- The aggregate function is then applied **to each group separately**.
- Every column in the SELECT statement must either be:
 - included in the GROUP BY clause, or
 - used inside an aggregate function.

Example

Table: Employees

department salary

IT	30000
IT	35000
HR	25000
HR	28000
Sales	20000

SQL Query

```
SELECT department, AVG(salary) AS average_salary
FROM Employees
GROUP BY department;
```

Output

department average_salary

IT	32500
HR	26500
Sales	20000

Explanation of Example

- Rows are grouped by department.
- The AVG() function calculates the average salary for each department.
- One result row is returned per department.

Q-2 . Explain the difference between GROUP BY and ORDER BY.

Ans :

GROUP BY

- **GROUP BY** is used to **group rows that have the same values** in one or more columns.
- It is mainly used **with aggregate functions** like COUNT(), SUM(), AVG(), MAX(), and MIN().
- It reduces multiple rows into **one row per group**.

Example

```
SELECT department, COUNT(*)
```

```
FROM Employees
```

```
GROUP BY department;
```

→ Groups employees by department and counts employees in each department.

ORDER BY

- **ORDER BY** is used to **sort the result set** in ascending or descending order.
- It does **not group data**.
- It only changes the **display order** of the result.

Example

```
SELECT name, salary
```

```
FROM Employees
```

```
ORDER BY salary DESC;
```

→ Sorts employees by salary from highest to lowest.

Key Differences

GROUP BY

Groups rows with same values

Used with aggregate functions

Reduces number of rows

Creates summary data

ORDER BY

Sorts rows

Used for sorting results

Does not reduce rows

Only changes display order

SQL View

Q-1 What is a view in SQL, and how is it different from a table?

Ans :

1. What is a View in SQL?

A **view** in SQL is a **virtual table** that is created using a SELECT statement. It does not store data physically; it only displays data taken from one or more tables.

2. How is a View Different from a Table?

Table

- A table is a **physical database object**.
- It **stores data permanently** in the database.
- Data remains stored even when not queried.
- Created using the CREATE TABLE statement.
- Requires storage space.

View

- A view is a **virtual database object**.
- It **does not store data**; it shows data from tables.
- Data in a view is always **up-to-date**.
- Created using the CREATE VIEW statement.
- Requires very little storage.

3. Example of a View

```
CREATE VIEW Employee_View AS
SELECT emp_id, name, department
FROM Employees;
```

Q-2 . Explain the advantages of using views in SQL databases.

Ans :

Advantages of Using Views in SQL Databases

Views provide several benefits in SQL databases by making data access easier, safer, and more efficient.

1. Simplifies Complex Queries

- Views store complex SELECT queries.
 - Users can retrieve data easily without writing long or complicated queries.
-

2. Improves Data Security

- Views can restrict access to sensitive columns or rows.
 - Users see only the data they are allowed to see.
-

3. Provides Data Abstraction

- Views hide the underlying table structure.
 - Changes in table design do not affect users if the view remains the same.
-

4. Ensures Data Consistency

- Views always display **updated data** from base tables.
 - No data duplication occurs.
-

5. Reduces Query Errors

- Since logic is predefined in views, chances of writing incorrect queries are reduced.
-

6. Supports Reusability

- A view can be reused by multiple users and applications.
- Promotes standardized access to data.

Rollback and Commit Savepoint

Q-1 Explain the concept of SAVEPOINT in transaction management. How do ROLLBACK and COMMIT interact with savepoints?

Ans :

Concept of SAVEPOINT in Transaction Management

A **SAVEPOINT** is a marker set within a database transaction that allows you to **roll back part of the transaction** instead of rolling back the entire transaction.

It is useful when a transaction has multiple steps and you want the option to undo only specific changes.

How SAVEPOINT Works

- SAVEPOINT divides a transaction into **smaller logical parts**.
 - You can return to a specific savepoint if an error occurs.
 - Changes made **before the savepoint remain safe**.
 - Changes made **after the savepoint can be undone**.
-

Interaction of SAVEPOINT with ROLLBACK

- ROLLBACK TO SAVEPOINT undoes changes **after the specified savepoint**.
- The transaction **continues to remain active** after rolling back to a savepoint.

Example

```
SAVEPOINT sp1;
```

```
-- some SQL operations
```

```
ROLLBACK TO sp1;
```

This cancels only the operations performed after sp1.

Interaction of SAVEPOINT with COMMIT

- COMMIT makes **all changes permanent**, including those before and after savepoints.
 - After a COMMIT, **all savepoints are automatically removed**.
 - You cannot roll back to a savepoint after a COMMIT.
-

Key Points

- SAVEPOINT allows partial rollback within a transaction.
- ROLLBACK can undo changes up to a savepoint or the entire transaction.
- COMMIT permanently saves all changes and clears savepoints.

Q-2 When is it useful to use savepoints in a database transaction?

Ans :

1. Partial Rollback Is Needed

- When only a **part of the transaction fails**.
 - You can roll back to a savepoint instead of undoing the entire transaction.
-

2. Long or Complex Transactions

- In transactions with many SQL operations.
 - Savepoints allow you to safely return to a known correct state.
-

3. Error Handling and Testing

- Useful when checking data conditions during execution.
 - If an error occurs, you can undo only the recent operations.
-

4. Batch Processing

- When inserting or updating data in batches.
 - If one batch fails, you can roll back only that batch using a savepoint.
-

5. Financial or Critical Transactions

- In banking or order-processing systems.
- Prevents loss of valid earlier changes if later steps fail.